

Okay, hello everyone.

Welcome back to the Deep Learning Systems Algorithms and Implementation.

So this is lecture eight.

In today's lecture we're going to walk you through on some of the detailed implementations of neural networks through our IPython notebook and it is going to cover a lot of the contents of specific implementations on what we discussed in the last lecture.

Again, this is going to be an implementation section, so I'm going to use IPython notebooks instead of slides to walk you through some of the specific content here.

As a first step, you will need to run this code block in here, so if you run it for the first time, you want to uncomment this particular code block that clones up these lecture 8 onto your local.

And then after that you'll be able to do some of the local modifications to continue right, and in the next time you can just comment this so that

you'll get kind of a persistent drive of Needle repo in here and we can take a look on the right hand side there is already Needle being cloned in here.

And this is going to be the code that we're going to use for today's lecture.

So if you have-- in the future we're going to release homework 2 and the same repo can be used.

You can also use this homework 2 repo for completing this piece of code.

Another thing that I want to mention today is that for today's lecture I do have a already implemented automatic differentiation of Needle.

So if you want to, while the lecture 8 repo do not contain implementation.

So you will need to copy and paste your own implementation of automatic differentiation onto the corresponding file in order to run some of these demonstration here.

Okay, so before we getting started, let's first do a bit of refresh of Needle since lecture four and homework one.

So we know that in lot cases we will be able to, you know, define a Needle tensor in here and here  $w$  is a Needle tensor.

And one of the things we want to quickly go over, is how can we go and update weight in neural network training?

And you know in all cases we talk about how can we forward neural network computations as well as taking gradient with respect to certain parameters.

But once we have those gradient, there's always a question of how can we go ahead and updates of weight values according to the gradient or according to some stochastic gradient update rules.

So let's try to do that, assume that we have a weight vector here as  $w$  and gradient vector here as  $g$ .

And one thing we can do is we can go ahead and first of all check the weight vector in here.

We can find that by default, when we create a Needle tensor in here, the require gradient field of this Needle tensor equals true.

That means that we are actually tracking the computational graph and gradient as we in order to compute gradient as we start to run some of computations.

So let's assume that we're going to run a stochastic gradient step there for, for several steps.

The one thing we can do is we'll be able to say take a gradient vector, we can take a learning rate and run this for loop here.

What this for loop does is effectively we're running stochastic gradient descent operations.

And one of the things that want to note in here is that I'm doing like negative learning rate So this is because the lecture 8 repo itself is, does not contain the subtraction operator.

But you know, if you have already implemented as part of your Needle repo, you will be able to actually go ahead and directly write  $w = w - \text{learning\_rate} \times \text{gradient}$ , as a stochastic gradient update.

As we talk about in last lecture, there's one problem with this particular implementation.

So what is the problem?

The problem is that every  $w$  itself actually contains a computational graph because  $w$  have this required gradient field being written here, right?

So, we can look at  $w.\text{operator}$  in this case  $w.\text{operator}$  correspond to elementwise add and we can also go ahead and look at  $w.\text{input}$

which is another Needle dot, which is another Needle value and you can find that also correspond to elementwise add.

You can even go ahead and trace, you know,  $w \cdot \text{input} \cdot \text{input} \cdot \text{op}$ .

So what's happening?

So what's happening is that you know in every iterations right, we are creating, so whenever we're doing these operations, we're creating a computational graph node that whose operation is elementwise add, right?

And then we are saying that we need to do  $w$  plus and this is learning rate times negative learning rate times gradient here and we do another plus operation here.

Another thing here, so effectively we are creating this train of computational graphs as we start to go ahead and write this rules.

And then in the end, the  $w$ , we kind of hold at the end of computational graph, right?

So of course in this case we do get the latest value of  $w$  but what is the problem of this approach?

The problem is that this particular  $w$  also takes a computational graph trace of all those previous computations, right?

So this is kind of like a common bug that sometimes people will encounter when you write programs in PyTorch.

Imagine that you're doing iterations over several, you know, you are doing things accumulating your loss across several iterations, then this kind of computational traces kind of accumulates.

As a result the memories of those previous gradient, as a result usually those gradient also contains like input computational graph to compute those gradient.

As a result, all those input computational graph get retained through this reference of  $w$  and they cannot get released.

So each iteration, the memory cost of the entire process kind of grow linearly as a number of iteration grows.

As a result you will get a out of memory error if you write code this way. And this is kind of like a one, a common bug that you can encounter while writing programs in like a imperative automatic differentiation framework.

So how can we avoid this?

So instead what we really want is, we don't really want to do this updates while keeping track of computation graph.

Remember that, why do we need computational graph being tracked?

The reason is that we need computational graph to be able to help us to perform the reverse mode automatic differentiation, right?

So because we know that we might want to compute the gradient with respect to some of the inputs.

So we want to be able to, you know, build up computational graphs to help us to, to track the reverse mode automatic differentiation.

However, for the weight updates, we know that most of the cases we don't care about differentiating with respect to the weight update rules.

So as a result we don't need to compute, we don't need to track combinational graphs in that setting.

Instead we can try to create what we call detached tensor.

In this case, you know, `w.data` is also a short, shortcut for `w.detach`, which create a detach tensor that corresponds to the same content but do not contain the corresponding computational graph attached to them.

So if you look at, you know, `w.data.requires_grad_` and it becomes false, right?

And that and this.

So, so what's what's behind us saying this data?

We can go ahead and open up the code of Needle and take a look at what's happening here.

So if you look at this file and we scroll down to the `w.data`, you can find that the data function simply calls `self.detach` where the detach function calls `Tensor.make_const`, `self.realize_cached_data`, okay?

So what's happening is actually is creating a new tensor that shares underlying cached data.

So it is not, we're not copying a new tensor, we're creating a new tensor that shares the same underlying memory with respect for original `w` except that we are removing all the additional fields such as inputs and op field from that particular data that help us to be able to, you know, if actually cut down the computational graphs.

So we can also go ahead and take a look in here, right?

So if you look at say, so if you look at here and I can try to say, `z.equals(w.data)` and let's check you know, `z.op`, you find that in this case they `z.op` actually is None, so you can verify that's true `z.op` is None so it does not contain op it does not contain inputs.

This input is empty, right?

So and if you look at, well it, it does contain a cached data field and you'll find that actually this cached data field actually is `w.cached_data` as well.

So they do share the underlying data presentations.

As a result we're not doing like a deep copy of memory, okay?

So in order to perform updates, what `w` can say is we can say instead, you know, now new `w` equals `w.data * negative learning rate` times `gradient.data`.

So if we run this, we find that we're going to get a new `w` value in here.

In this case because both `w.data` and `gradient.data` are detached, no longer attached computational graph to them.

So if you look at required gradient field of this new `w`, you can find that it corresponds to false as opposed to true.

So the rule that we're having here is actually if you are in eager mode and we are doing arithmetic computations.

If all the inputs do not require gradient, then by default we will not construct a attached tensor that contains computational graph.

So automatically create a detached tensor.

As a result we can find new `w.inputs` also equals empty.

So we can also go ahead and track the code that leads to this piece of logic.

If you look at here, you can find that, let me see if it's `tensor.make` this is value and there is also `tensor.make` from op.

So in here if it's not in lazy mode and if in the end if tensor does not require gradient, usually if tensor doesn't require gradient or initialize it,

we'll check if all the inputs require gradient, if any of them require gradient, then this tensor will require gradient.

If it does not require gradient, what we can do is we can simply create a detached version of the tensor that effectively runs the computations and ignore the previous attached computation graph.

So why does detached is more preferred in this case?

The reason is that in a lot of cases if you attach the computational graph along with your computations, you hold references to those previous computational graph node or those value node.

As a result, these value that wouldn't get released until all the previous reference to that value get released.

So in this case we could end up having a long chain of references where a lot of memories that could have been released didn't get released beforehand.

So as a result, when you are trying to build some of you computations, always ask yourself whether or not you can try not recording computational graph.

In PyTorch there's also a context of stop gradient or no gradient that serves similar purposes.

So besides creating a new  $w$  is actually helpful to directly try to update  $w$ 's content in an inplace way.

This is because in a lot of cases, you know the  $w$  is being already referenced as we will see in the later part they're being referenced as part of the parameter fields.

So it's helpful to directly go ahead and update their values as opposed to create a new variable that then gets updated onto the corresponding content.

So in order to do that we can do  $w.data\_equal$ , so in fact you can try to have a setter that sets onto  $w.data$  and in this case simply copies the corresponding data content onto the new  $w$ .

And if you look at  $w$  again, you'll find that in this case, you know the the  $w$  actually indeed correspond to the new updated weight, right?

So this is the way that we normally use to write our weight update rules.

Okay, so now we have talked about the ... of Needle and how we can go ahead and use this in place mutation.

We can try to be able to take a detached tensor, perform operation on them, which again creates a detached tensor, and in place they mutate some of the weight variables in here.

As the next step, let's start to take look at another thing that's going to be helpful in your, in building your homework 2 and building some modules.

They have to do with the numerical stability.

So in one cases, you might have noticed already, like if you look at here, you can find that after several steps of update, we're getting these values like 0.3999992, right?

While in fact, every time all we are doing is we're trying to subtracting 0.1 here.

So effectively it should have been 0.4.

In this case you should have been 0.4 as opposed to 0.39992.

So why does that happen?

This is because we are not, when we're storing the data on memory, we're storing them through a format called floating point number.

In this case the floating point number have limited precisions, right?

So usually a floating point number contains two parts.

There's a what called mantissa part and times two to the power of exponential part.

So effectively, if you have a floating point number, you will have, there's sign bits in here and you have your mantissa exponent in here.

Because you know you have limited number of bits, to represent the mantissa value and the exponential value, as a result, you cannot expect to express every real number in a precise fashion.

That's why when we're doing arithmetic operations, in a lot of cases you kind of get this arithmetic numerical error when running computation.

So imagine that we want to perform softmax, which is commonly used operations in deep learning.

We are also going to use that in some of our homeworks.

Sometimes it might involve a small variant like log softmax as opposed to softmax.

So in order to compute this, of course one of the easiest ways we can go ahead and say, "Hey, I'm going to directly compute this."

So the softmax being written as formulas, we're going to take exponential and going to normalize by the sum of all of it.

So easiest thing we can do is we can naively follow the formula to compute the softmax.

For example, if I'm going to import numpy here and imagine that I'm going to have input array, it really depends on what the value of input array is.

If the input array is like 0, 0, 1 going to do softmax naive here.

In this case, you can find that effectively the code first does exponential, right, then normalizes,  $z$  divided by the sum of all the values, it gives you the right result.

So instead what if we putting a bigger value here?

For example, if you're putting 100, 100 and 101, if you run this code again, you find that we're getting not a number in here.

So what's happening here? Again, it is because the floating point number have limited precisions and if we go ahead directly take exponential with respect to a hundred, this value is going to explode.

Because we, with a number of bits available, we are really not able to express this number in the memory.

So what can we do to avoid this case?

And there's approach that we can do to avoid this kind of numerical error.

And the way we do it is, you can find that actually there's an invariant in here.

So if you have exponential of  $x_i$  and if we're subtracting each of  $x_i$  by a constant values  $c$ , you'll find that actually the computation does not change.

However we can leverage this fact, to try to scale, try to actually change the value of  $x_i$  so that they become a small value as opposed to a very big value

because we're taking exponential of something, say smaller than zero.

Usually it's much more accurate than taking exponential of a very big number.

So we can do is we can try to pick  $c$  to be maximum of all those  $x$  so that we'll be able to, actually after this transformation all the values are going to be negative except one that's going to be zero.

So in order to perform this, what we can do is we can try to take  $x$  and we're going to subtracting  $x$  by the maximum values in here,

that's performing this transformation, right?

And then we're going to run the same exponential normalization again and if we run this and then can provide a very big number here and what you find that is actually gives us kind of a stable result, it's not no longer going to overflow.

So similar principles applied to other operators like logsoftmax operator and logsumexp operator.

Both of them are quite widely used as part of normalization process in our end to end flow.

So this is the stable computation of softmax and that is the approach that we can use to implement some of homeworks.

Of course in certain cases, in this case we kind of need to be aware of the fact that we are computing softmax,

and we are providing a numerically stable implementation of that softmax operations.

For advanced computational graph system, we might be able to also go ahead and detect computation where such kind of overflow can happen

and try to automatically re-normalize this to enable stability, but that's kind of an advanced topic.

Maybe you can take as part of our project and we wouldn't require that as part of our homework implementation.

So we've talked about some of the basics and now let's start to talk about how we can design a neural network library on top of our current set of implementations.

Remember that as part of our homework one we're only implementing a tensor class as well as arithmetic computations along the tensor such as we can do addition, subtraction.

We're also able to do things like, doing automatic differentiation or backward function which is what you implemented as part of our homework.

In the last lecture we also talk about at high level what are the important components of the neural network library.

So we know that there's a module that can help us to compose our models.

We can also use a module abstraction to build loss functions.

We will need to be able to help us to initialize the weights using different initialization functions.

It's optimizer, that specifies how do we take parameters and the gradients and that's also data loading and pre-processing.

So today we're going to go ahead and implement some of those.

In particular we're going to focus on the nn. Module.

So let's start with module anyways and I'm going to do a bit of coding together with everybody here cause it's coding can be fun.

So in order to get the module, the first thing that we need to get is this parameter class.

We kind of already have it here.

So we have this parameter class.

So in this case this parameter class is a sub class of tensor and the only reason why we need this parameter class

is we just need a way to indicate this is a special kind of tensor, which is called parameters.

And then we want to have a way to be able to say that in certain cases want to be able to say that w is instance of parameter,

so in fact parameter have nothing special about parameters and you don't even have to specify the constructor of it,

it's just the way to know that hey this is special kind of tensor called parameter.

Now with this special kind of tensor we are ready to introduce a module class.

So a module class contains a few things, the most important function of module class, this function called parameters

and it defines how are we going to go ahead and pull all the parameters from this particular module.

And in order to do that we need to define an auxiliary function called `_get_params` and that contains a value.

And I'm going to implement this and I'm going to walk you through this after I finish the implementation.

The first thing I'm going to check is if `isinstance` value, parameter, if it's a case I'm going to directly return.

Then the second case, if `isinstance value, dictionary`, in this case we're going to go ahead iterate over all these items on this dictionary and return `_get_params`.

So we are going to say result equals a list.

Result plus equal `_get_params v` and we're going to return `v` here.

And there's a third case where actually we're not going to return, we are going to return `res`.

The third case is we're going to check if `value` is instance of module and if it's a module, we are going to return `value.parameters`.

And in here what we are going to do is we're going to do return `get_param self.__dict__` So this `dict` is a special attribute of a class.

It tries to gives you every member of this class.

And so effectively what's happening in here is that we want to allow these parameters to be able to pull up all the classes that are subclass of parameters and return that to the inputs.

What we can find what it does here is try to recursively look into the value, call `get parameters` and then trying to be able to find the corresponding parameters and return them as the list.

We can go ahead actually try to create some modules to see what is the effect of it.

So let's assume that I want to say create what we call `ScaleAdd` which is a subclass of module.

In in this `ScaleAdd` modules, we going to define two parameters in here.

The first parameter is going to be as `s` vector.

So let me define `s` equals one, initial `b` equals zero.

Let's say we have `self.s` equals parameter.

Init `s`.

`Dtype` equals `float32` and I have `self.b` equals parameter init `b`, `dtype` equals `float32`.

And then, okay, so in here, besides the parameter function actually we'll also to want to define a call function which simply redirect it to `self.forward`.

So after running this and in here I can define a forward function as input `x`.

What we're going to do is we're going to return `x` times `self.x` plus `self.b`.

We're going to find that this is effectively one neural network module that can create and we will be able to create one instance of it by calling `ScaleAdd`.

And there a few things we can do with this module.

So the first thing we can do is we can say `ScaleAdd.parameters`.

If we look at this you can find a returns a list of two tensors.

As a matter of fact this `ScaleAdd.parameters` correspond to this two field, `self.s` and `self.b`.

And so why does the parameter return this?

We'll look at the implementation and you can find that effectively the `self.dictionary` of that `ScaleAdd` contains `s`, that `s` value, right?

And `b`, that `b` value in here we get this dictionary being passed onto this function that goes into this code parse that unpacks it and calls `get_params`.

And for both cases the value itself is a parameter.

So it gets pulled up.

If I have other values that are not parameters, they wouldn't get pulled up to the output in here.

So this parameter function is a very convenient method for us to pull out recursively all the fields that are corresponding to parameters.

You'll even be able to go ahead and check when you will be able to say that this zero is sadd.s here, which is true and is 1 is sadd.b.

So you can find that indeed this parameter corresponds to this two fields, the s and b.

Another thing we can do is now we have this module so we'll be able to go ahead and try and apply it.

Tensor 2, dtype equals floating point 32.

If you apply it you can find that the value, it's going to be another Needle of tensor.

Let's see.

Okay, so this should be return, right?

So I forgot this return value in here, go ahead and try again.

If you run this, you'll find that it's going to return your Needle tensor.

In this case because scale initially is one so it's going to give you the same tensor but if I parameter initial has to be two, run again you can find that equals four, if I parameterize by two one, it's going to give me five.

So, so you know effectively for module there are two things we can do.

We'll be able to recursively query what are the underlying parameters that are contained in that module.

The second thing we can do is going to go ahead and take some of this input tensor and apply that and gives you the output tensor.

So now we have defined a scale add, another thing that we can do is we will be able to actually recursively define modules with respect to other modules and that's a very useful ability in here.

Let's imagine that want to define something called MultiPathScaleAdd which also module and effectively what it try to do is try to have two individual pathways.

So we have inputs here x that's going to go through a ScaleAdd pathway and another ScaleAdd pathways and at the end we're going to add these two things together and that's my output.

So this is my multi-pathway ScaleAdd.

Of course you know realistically it doesn't mean anything here we're only trying to use this a demonstration of

how you can go ahead and compose module together and you will be able to use the same way to developed residual module in your homework.

So in order to do that, the first thing we can do in here is we'll be able to define your constructor class.

In this case we have two paths, zero equals ScaleAdd, self.path1 equal ScaleAdd, so we kind of get two path and in a forward function

we can say that I want to return self.path0 applied to x plus self.path1.

That's our MultiPathScaleAdd and we'll be able to say mpath equals MultiPathScaleAdd here.

And again in this case I will be able to try mpath.parameters.

For all this you can find that we're getting four tensors in here, right?

So even though in this case this module do not have immediate field that contains a parameter, what they try to do is actually when you call

`mpath.parameters` because we know that this module contains some other submodules, `self.path0` and `self.path1`.

So recursively call the parameters of `self.path0` and `self.path1` to get their parameters.

As a result we are collecting all the parameters of this module recursively and this is going to be helpful when we are going to introduce optimizers and go ahead and update the weights in here.

Okay, so, and we can also go ahead and try, so in this case actually this one, the first parameter we can go ahead and track what it corresponds to.

It actually corresponds to `mpath.path0.s` and second parameter, corresponds to `mpath.path0` and so on.

So now we have this, we will also be able to do things like `mpath` directly apply it on `ndl.tensor` one, `dtype` equal floating point 32, that give us the corresponding output.

In case we have two passes adding together, it's two.

So now we have defined all this, right?

The next thing we can go ahead and try to do is to define a loss function because once we have the modules we want to be able to compose that together with loss function so we can perform gradient update.

So in order to define loss functions let's, we know that loss function can be defined as a module.

You already see implementation in here.

So in this case loss function is some module that does not contain any parameters.

In this case, this particular loss function, we're taking two things, we're taking input, which is the output prediction of my overall pathways,

as well as label `y` that corresponds to the loss value and the way we compute loss function is we're going to go ahead and compute effectively

the difference between, again the negative one is artifact of us not having subtraction operators.

If you have a subtraction operator, you'll be able to go ahead directly say `z` equals `x` minus `y` and yeah you are returning this square.

So in this case we are simply defining `L2Loss` that computes a difference between two values.

Once we have a loss function, we'll be able to write `y` equals a label in here.

And then in this case, right, let's say `x` equals two and you've applied full `mpath`, the end result is four.

And then let's assume that we have label, which is 2.

What we can do is we can go ahead a first compute `mpath` that gives us output of the hypothesis and then we're going to use the loss function and compute the loss value.

So go ahead and run it.

You'll find that the loss value equals four because the forward prediction of a value equals four and then the expected value equals two, right?

Difference of the square there, the value equals four.

And if you'll go ahead and do loss now backward, we're going to get the gradient values of each and actually we'll be able to go ahead and verify that.

So we can go ahead and launch since like `parameters` equals `mpath` of parameters.

In this case actually you know each of parameters will come with the gradient view.

So we'll be able to say that `parameters`, `params[0].grad`, that equals eight.

And if you look at actually this also effectively is the path0.

scalar . gradient, right?

So why is equals eight can go ahead and compute the loss.

So because we know that it's, the loss value is four, And then if you think about how we're going to backpropping that the gradient values with respect to x in this case is going to equal.

Let's see, first of all, the gradient value of a loss is going to equal two times z.

In this case the value z equals two in here.

So as a result, the gradient value with respect x equals four.

Let me see if that's right.

Oh, it's really hard to compute this manually.

Let's see the loss value equals four because this square, so the gradient value equals two times two, which also equals four.

And is the gradient value right?

Oh yes.

And because my input is x is two, right?

So again, it's four times two.

So the gradient value equals eight, okay, so actually we are indeed get right gradient.

Yeah.

So once we have this gradient, we'll be able to go ahead and take a loss function, write up this optimization rule, we'll be able to say that for i j parameters,  $w.data$  equals  $w.data$  plus minus rate times w of gradient.

You can find that one thing that's really convenient about in place update is that you know, every time we are doing updates with directly set gradient

values back onto to w and those w's are still contained right within the original module.

So the second time when we call forward, they're still going to be being there the same with the same w fields.

So we don't need to be worry about how do we pass the new weight value back onto the model.

Okay, so now we have talk about modules.

So next thing we're going to talk about is how we can define an optimizer interface and an optimizer kind of help us to be able to perform updates like stochastic gradient updates and other updates in here.

And there are two important functions of an optimizer.

The first function is called a reset gradient.

So first of all, an optimizer is going to take in a list of parameters that effectively get passed from `module.parameters` here.

So once we have those parameters, there are two things we can do.

The first thing is reset gradient.

So it will simply iterate over all the parameters and reset the `p.gradient` field to be None.

The second function is going is called step function.

So in this case it iterates over all the parameters will take the all the weights and perform one step of updates in here.

Either stochastic gradients in or stochastic gradient with momentum depending on the update rule we are looking at here.

So we can go ahead and try to implement a variant of a variant of optimizer in here.

So let's imagine that we have, let's go ahead and implement simplest we do SGD is a subclass of optimizer

and we'll be able to say that init function is going to be, there's a learning rate and yeah it's a parameter in here

and then it's going to record, well I'm going to call my parent's initializer, right?

So super optimizer, I kind of forget what's okay, so actually I can directly do initialize, that's fine because in this case the initializer is not as,

so I can just do `self.params equals params`.

`Self.learning rate equals learning rate`.

And then in a step function we can do for `w` in `self.params`, `w.data equals w.data plus minus self.learning rate times w.grad.data`.

So this is going to be the stochastic gradient optimizer.

We need to define optimizer first here.

Okay, so now we have implemented this SGD optimizer and next thing we can try to do is try to put everything together,

try to run things end to end to see how does things work together.

Okay, so in order to do that.

What we can do is we can try to build this code block that have been written here.

The first thing that we can try to do is we can try to create two `Tensor` tensors where `x` correspond to my input and `y` corresponds to my label tensor.

And then we can create a model that correspond through my hypothesis class that correspond neural network module.

In this case, I'm simply using a multi pathway scale add.

In this case in the loss function, in here we're going to use a `L2Loss`.

So next what we're going to do is we're going to create a optimizer.

In this case, SGD is going to be the optimizer that we're going to use and then we're going to run it through a number of epoch here.

So in each of the epoch, what we're going to do is we're going to first call `optimize reset gradient` and then we're going to call the `hypothesis.model`.

The model `x` that gives you a hypothesis and then we're going to create a loss function that computes a hypothesis and label that gives us a loss.

And effectively this is a training loss that we can go ahead and print it out and we're going to call `loss.backward`.

That gives the gradient.

Finally `opt.step` gives us the one step of SGD update so we can go ahead and try to run this code block and you can find actually, loss decreases in here.

Which means that the indeed there is right amount of training epochs to help us to be able to build those end to end neural network training.

So I'm going to pause here a bit.

I will ask everybody to think about, what is the difference between this piece of code versus say the code that you have been written in your homework zero

where we are kind of effectively derive the gradient from hand and write up the update rule in C++ by scratch.

If you look at this particular piece of code you can find it's very modular, right?

So we'll be able to, imagine I want to change my hypothesis, I just need to change list line without changing any other parts, it's going to work out a box.

If I'm going to change my update rules like SGD?

I can just go ahead and change this particular line and optimizer it and the same code structure is going to continue working for a lot of cases and if I'm going to change my loss function, I'm going to have to change this line.

So this code is very modular, it help us to be able to replace different components as we need and that's kind of like the amazing part about deep learning framework

it gives us the ability to be able to mix and match different modules together.

And one reason is that we're designing those modules in a way so that they can effectively interact with each other organically.

And if you come back and look at those designs, you can find, hmm, those designs look so obvious, right?

They are so simple and elegant. On the other hand, if you put yourself into the time where this design does not exist yet or in the

first generation different framework, you may find that actually they are not as obvious at it seems, and it's really important to appreciate this kind of modular design and how they compose together.

One of things I want to notice that we only implement SGD, right?

So on other cases we will need to be able to create states in SGD or in optimizer as well if we want to implement something at SGD with momentum.

In this case we will not only need to record the set of parameters, but we also want to be able to create the additional states in here, right?

So that we can keep track of momentum variable.

So for example, we will need to be able to write things like `self.momentum`, let's use `u` to denote momentum it's equals, `Needle dot tensor0`.

Actually I can do things like `Needle dot zeros` like `p` or `p` in `params`.

So it will help us to create a set of momentum variables.

Then we can use in our update rules.

So when we start to create more advanced optimizers, we want to be able to keep track of those internal states within the optimizer class.

And that's also one of the interesting part about the optimizer design where we have a module that contains the parameter states,

we have the parameter function that collects out of the list of parameters and then we use this, we recreate those additional auxiliary states that help us to run the corresponding updates.

Okay, so that's end to end flow.

In next part I'm going to quickly go over initialization.

So in this case, as part of homework we need to implement initialization function for weight, seeing things like linear transformation,

other transformations in order to derive the overall magnitude of the random initailizations.

One things that we need to remember it's a very deep linear neural networks, every time when you apply a transformation the all magnitude of the value can change according to the weights.

And one of the goals in here is we really want to derive in initialization such that your, if you are going initialize randomly and your input is,

also randomly distributed, your output varies, it's going to be close to your input variance and there's a derivation from the original paper

that effectively says that my output variance, it's going to equal the second order two-norm of inputs of each of input and a variance of  $w$ .

In this case let's assume that we're doing Gaussian initialization of  $w$  the variance equals sigma square.

And also let's imagine that if  $x$  square is  $x$  as a result of a ReLU activation of previous layer, we know that the result of the variance of a relu

because you know you get kind of like the original value on right hand side and zero on left hand side.

So we kind of get half of variance in there and our goal is to make sure that the variance of the next layer equals a variance of previous layer.

So we plug that back and derive the sigma, you'll find that you can have a value to set the sigma according to a number of input fan lengths of your input width.

If you change your distributions to things like uniform distribution, we will need to use a similar rule except that you want to use the variance calculations for uniform distributions.

So we have talked about the overall modules and initializations and how does loss function and everything fit together.

So the last part I'm going to quickly note about, it's kind of a bonus content on tuple value.

So if you open up your code on the right hand side, actually the lecture 8's code now contains additional implementation of tuple, of a tensor tuple.

Let's see, of a tensor tuple that corresponds to a tuple of tensors.

In this case we're not supporting nested tensors as well as some operation that correspond to tensor tuple such as getting value from tuple and concatenating tuples together.

It also allows us to be able to define operators whose return value are tuple values.

So if you look at ops in here, actually we have this operator called FusedAddScalars.

Effectively it's ... in the compute it's going to return two value as opposed to one.

And output gradient, actually in this case it's a tuple and in our case we want to, because  $a$  is present in both cases, we're going to add those two gradient together.

So one thing you'll find is that with the same autograd mechanisms we'll be able to create a tensor, run it through a tensor operator

that gives you a tensor tuple and actually continue to give you tuple values and then you know you'll be able to obtain the elements of the tuple values.

So you are more than welcome to check it out and you know it is a very useful way for us to express, say operation that contains multiple return values

and the same mechanism you, it is amazing to see that same automatic differentiation actually works out of box in here.

So for example, I would be able to say that  $v_0$  equals  $z[0]$  plus  $z[1]$  times two and  $v_0.backward$  and it should be able to, backward, they should be able to propagate the gradient back onto  $x$  properly.

And if you are more than welcome to go and check it out and if you're interested or perhaps not part of the requirement, you are more interested, you are more than welcome to implement and say automatic differentiations for other data structure values like dictionaries.

So that's the end of this lecture.

Thanks everybody for coming and I will see you folks in next few lectures.